INTENT BEFORE ACTION: DECODABLE AND STEERABLE SUB-GOAL REPRESENTATIONS IN LLMS

Anonymous authors
Paper under double-blind review

ABSTRACT

When a language model writes code for a multi-step task, what does it represent about the operation it is about to perform? We find that information about the next subgoal is available before the model emits its implementation. We localize a small set of attention heads using cross-language activation patching and introduce GOALSCOPE, which uses their output contributions to elicit a natural-language description of the upcoming subgoal in a separate model run. The same contributions can steer a new code context to implement the source subgoal in another programming language. Small head sets support effective readout, while a broader set is causally important for native implementation: preserving its access to task information largely preserves performance, and restricting its attention sharply reduces performance relative to random-head controls. Query and key/value interventions further show that these heads support goal-specific selection and adaptation to changed task information. An OV-based readout traces the availability of subgoal information through model depth. We replicate readout and implementation-transfer experiments within multiple models and explore readouts beyond explicitly specified code tasks. Together, these findings identify a readable, transferable, and causally relevant representation of an upcoming subgoal, without presupposing a particular algorithm for selecting it.

1 INTRODUCTION

A request to process a list of strings can involve several operations: remove invalid entries, retain palindromes, and count the remaining items. Before writing the code for one of these operations, does a language model already represent which operation it is about to implement? We study this question at the level of a *subgoal*: a meaningful intermediate operation that can admit multiple implementations. Our use of *intent* refers to the identity of this upcoming operation, rather than to a subjective state or a complete plan for the program.

Autoregressive prediction is compatible with representations of future computation. Prior work finds information about future tokens and constraints in model activations (Pal et al., 2023; Lindsey et al., 2025). Here we ask whether an upcoming subgoal is accessible as a semantic representation before its implementation appears, and whether transferring that representation can guide computation in a new context. Code provides a controlled setting in which the intended operation can be specified independently of its syntax and evaluated across programming languages.

Our main experiments use `Qwen2.5-Coder-32B-Instruct` on sequential predicate-implementation tasks. We identify attention heads whose output contributions transfer a predicate between contexts with different programming languages. We then evaluate these heads on multi-step prompts at positions around the beginning of the next implementation. The paper develops three findings.

Upcoming subgoals are readable and transferable. GOALSCOPE extracts selected-head contributions at a source position and injects their aggregate into a separate model run with a fixed verbalization prompt (Section 3). Its continuation describes the upcoming operation. Injecting the signal into a code stub instead tests whether it can induce implementation in another programming language. These complementary readouts test semantic accessibility and behavioral transfer; neither requires the source code to be supplied as text to the probe.

Sparse readout and native causal involvement differ. A small head set recovers much of the readout performance of the full selection, but a broader intervention is needed to strongly disrupt native implementation (Section 4). We distinguish these findings with controlled span-restriction and sink-attention experiments. The results identify a causally important route for task information without claiming that the selected heads alone constitute the entire subgoal computation.

Subgoal information is available in both attention and the residual stream. The heads attend to relevant task descriptions, and changing their task-span keys and values can redirect implementation. Query transfer can also favor a particular source subgoal (Section 5). An OV-based transformation makes subgoal information readable from intermediate residual states. Together, these results constrain how the representation interacts with context while leaving its precise computation open.

We also examine few-shot relation tasks and free-form code to explore the reach of the code-localized readout (Section 6). Replications within other model families and training regimes are reported in Appendix L.1. These are replications of the phenomenon within each model, rather than transfers of activations between models. Our central claim is that an upcoming subgoal has a readable and manipulable representation before its implementation, not that the model forms its entire plan in advance or represents that plan in natural language.

2 Task and Intervention Setup

2.1 Language models and head contributions

An autoregressive transformer M maps a token prefix $x = (x_1, \ldots, x_n)$ to a distribution over its next token. Write h_t^ℓ for the residual state at layer ℓ and position t. Suppressing normalization and positional encoding, a head's contribution to the residual stream is

$$a_t^{\ell j} = \left(\sum_{i \le t} \mathrm{Attn}(q_t, K)_i v_i \right) W_O^{\ell j}, \qquad \mathrm{Attn}(q_t, K) = \mathrm{softmax}\left(\frac{q_t K^\top}{\sqrt{d_{\mathrm{head}}}} \right). \tag{1}$$

Here queries, keys, and values are learned projections of the normalized residual states; queries and keys additionally receive the model's positional transformation. In grouped-query attention, multiple query heads share a key/value head. The output projection maps each head's attention-weighted value into residual-stream coordinates. We write $[\ell, j]$ for a head and use *head contribution* for this projected output, rather than for its attention weights or raw value states.

2.2 Sequential subgoal implementation

A prompt specifies an ordered sequence $G = (g_1, \ldots, g_K)$ of independent predicates and asks the model to implement them sequentially. Predicates operate on integers, strings, lists of integers, or lists of strings. The main evaluation uses a locked corpus of 253 specifications. We prefill completed predicates and an assignment prefix for the next predicate, which defines the *frontier* subgoal g_k. This controlled frontier supplies the target description for evaluation; it does not guarantee that every clean completion implements it correctly.

We inspect five positions around the frontier: the end of the preceding line, indentation, the variable name, the assignment operator, and the first implementation token. We call a pre-implementation capture position a *pivot*. The assignment operator is the principal pivot in the ablation experiments. The first implementation token is a comparison position and is not evidence of encoding before implementation. Thus the corpus provides 253 specifications and up to 1,265 position-level records; the five positions are repeated measurements of each specification. Exact tokenization is described in Appendix C.1.

Progress denotes the completed subgoals. It is equivalent to an index in G only under execution in the specified order without skipping. We use this distinction when changing task order or the prefilled work. Our object of study is information about the next subgoal, not a presumed representation of a complete execution plan.

2.3 INTERVENTIONS AND GENERATION

Activation patching replaces selected activations in a destination run with activations cached from a source run (Vig et al., 2020; Mueller et al., 2026). We write $M(p)[a_t^{\ell j} \leftarrow a_{\text{src}}^{\ell j}]$ for an intervened forward pass. For multi-token generation, the intervention additionally has a schedule specifying its prefill positions and subsequent decoding passes. A fresh patch applied at every decoding step differs from a single prefill patch whose effects persist through the cache.

We distinguish three interventions. *Head-output replacement* substitutes individual head outputs and is used for localization and replacement readout. *Aggregate injection* sums selected projected contributions and adds the resulting vector to residual states in a separate probe; this is the main GOALSCOPE readout. *Query or key/value transfer* changes, respectively, the query states or task-span keys and values accessed through selected heads. These latter interventions probe sensitivity to goal selection and task information (Section 5). A resulting output change establishes an effect of the intervention; its interpretation depends on the source/destination contrast and the sites and duration patched.

3 READING SUBGOALS BEFORE IMPLEMENTATION

We first ask whether selected activations contain information that can describe and guide the upcoming operation before its implementation appears. We identify heads using a simple transfer task and then evaluate their readouts on sequential subgoal implementation.

3.1 LOCALIZING GOAL HEADS

A source and a destination prompt request different predicates on the same datatype in different programming languages. Both end where the next token distinguishes their respective implementations. We cache source head outputs and learn a sparse mask that promotes a target token associated with implementing the *source predicate in the destination language*:

$$a_{-1}^{\ell j} \leftarrow m^{\ell j} a_{\text{src}}^{\ell j} + (1 - m^{\ell j}) a_{-1}^{\ell j}. \tag{2}$$

The localization contrast uses a target token different from the source's next token, reducing the possibility of selecting heads solely for copying that token. This is a localization objective, not itself a guarantee of full semantic transfer. We call the selected set H the *goal heads*. The main readout uses 122 heads concentrated in layers 28–55 of the 64-layer model. Mask learning and head-set construction are detailed in Appendix B; subsequent multi-token evaluation tests what their outputs carry beyond the localization token.

3.2 GOALSCOPE AND IMPLEMENTATION TRANSFER

GOALSCOPE builds on activation-based verbalization (Ghandeharioun et al., 2024). At a chosen source token, we cache the selected heads' projected contributions and sum them into one vector. A separate run of the same model receives a fixed prompt asking what operation it is performing. We inject the aggregate into this probe's residual stream and greedily decode a natural-language readout. The probe receives neither the source instructions nor the source code as text.

Our main configuration uses norm-relative additive injection: at layers 35–45, the added vector has magnitude 0.1 times the local residual norm and points along the selected-head aggregate. The same source vector is applied at the probe's last prompt position and each subsequent decoding step. Appendix C gives the formula, prompt, and replacement variant. The source vector remains fixed during each readout; the probe's output is a multi-token generation.

We evaluate two complementary capabilities. **Readout faithfulness** asks whether the verbalization describes a nontrivial part of the frontier subgoal. **Implementation transfer** uses a code stub in another programming language as the probe context and asks whether its continuation implements a nontrivial part of that subgoal. Both employ interventions, but they test different outcomes: eliciting a description versus eliciting code. Native causal involvement is tested separately in Section 4.

An oracle judges readouts against the target description. Implementation-transfer scoring rejects trivial sign/zero continuations that can arise as defaults of the probe stub. A positive score denotes nontrivial agreement, not necessarily a complete or executable implementation. We retain clean

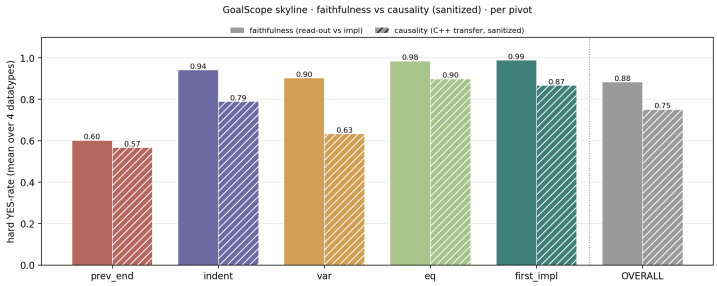


Figure 1: Selected-head readouts across capture positions on the OVReader evaluation corpus (64 examples per datatype). The first implementation token is shown as a comparison, separately from pre-implementation positions. The plotted "causality" metric is implementation transfer with trivial-readout sanitization. This corpus is distinct from the locked 253-specification corpus used for the pooled rates in the text.

generation as a separate reference rather than treating the intended frontier as proof of what the model actually computed.

3.3 Readout across capture positions

Across the locked corpus and all five positions, the 122-head readout achieves 87.9% faithfulness and 79.2% implementation transfer. These pooled rates include the first implementation token; evidence for *before implementation* comes from the position-wise results. Readouts are strongest near the assignment operator and first implementation token and weaker at the end of the previous line. The positional variation places the claim near the start of the subgoal rather than implying that the entire future program is represented from the outset.

Cross-language implementation transfer supports an operation-level interpretation: the same extracted contributions can guide different surface realizations. It does not establish invariance to every implementation choice. Similarly, a natural-language readout establishes that the signal can be verbalized, not that its native format is language. We next distinguish effective readout from the heads' causal role in ordinary code generation.

4 Sparse Readout and Causal Involvement of Goal Heads

A compact set may suffice to expose subgoal information without containing every path used for native implementation. We therefore separate readout performance at different head budgets from interventions that restrict the model's access to task information.

4.1 Small head sets support effective readout

Twenty-five selected heads, approximately 0.98% of the model's 2,560 query heads, recover 79.1% faithfulness compared with 87.9% for the 122-head anchor (Table 1). Larger sets do not yield a monotonic improvement. This establishes that small selected sets can support effective readout, without identifying a minimal circuit or implying that arbitrary sets of this size suffice. Comparisons with aggregate attention output and raw residual injection appear in Appendix L.

4.2 Preserving or restricting task access

We evaluate native frontier implementation on the same 253 specifications at the assignment pivot, with interventions continuing during decoding. Each example has a clean baseline, a goal-head condition, and a random-head control. We test the core 122 heads and a broader 250-head set.

Conditional sufficiency: preserve selected access. For heads in layers 20 and above outside the spared set, we block attention to the span from just before the frontier description through the end of the prompt. Spared heads retain access. Other key positions remain available, layers below 20 are intact, and all MLP computation continues. Thus this intervention tests sufficiency of *selected*

Heads	Readout faithfulness	Implementation transfer
5	.446	.425
10	.715	.625
25	.791	.779
50	.817	.839
100	.867	.813
122	.879	.792
150	.859	.816
200	.880	.794
250	.852	.750
300	.874	.779

Table 1: Readout from selected head sets on 253 specifications, pooled over five capture positions. All use norm-relative additive injection with $\alpha = 0.1$. Selections are not strictly nested and differ in their construction, so this is a comparison of available head sets rather than a controlled effect of head count alone.

Condition	122 heads	250 heads
Clean baseline	.925	.925
Spare goal-head access	.806	.913
Spare random-head access	.091	.095
Restrict goal heads to sink	.854	.158
Restrict random heads to sink	.925	.921

Table 2: Native frontier implementation under span restriction and sink-attention interventions ($n = 253$). The two experiments manipulate different access patterns; "spare" does not remove the rest of the network.

upper-layer access to the blocked span, not sufficiency of these heads in isolation. Random controls spare a comparable number of heads from the specified depth pool; the pool and exact blocked span are detailed in Appendix M.

Necessity test: restrict selected heads to the sink. From the pivot onward, we instead restrict the selected heads to attend only to position zero, while other heads operate normally. This removes their normal content-dependent attention but is not literal output zeroing. We compare with restricting an equal-size random set drawn from the depth pool and disjoint from the goal heads.

Preserving goal-head access retains substantially more implementation performance than preserving random-head access (Table 2). Conversely, sink-restricting 250 goal heads sharply reduces success, whereas the random control remains close to baseline. Restricting the 122-head core has a smaller but nonzero effect. These results establish causal involvement of the broader selection under the tested intervention. They do not prove a unique bottleneck, identify the computation that first selects the goal, or demonstrate active rerouting after ablation.

The combination is informative: a small set supports effective readout, while native implementation remains robust to restricting the core and is much more vulnerable to restricting the broader set. Readout sufficiency and the extent of native dependence are different properties of the representation.

5 Goal Information in Context and Model Depth

We next examine the information accessed through goal-head attention and its availability in intermediate residual states. These experiments characterize sensitivity to context and goal selection without presupposing a progress-subtraction algorithm.

5.1 Attention to relevant task descriptions

At a frontier position, the selected heads' value-weighted attention often concentrates on the task tokens describing the upcoming predicate. This resembles gaze behavior observed in other generation

settings (Gandikota & Bau, 2026). We use this alignment as a diagnostic of which instruction information the heads access (Appendix D). Attention alignment alone does not show that the model will implement the attended goal, or where that goal was selected.

5.2 Changing task information redirects implementation

Consider a source with tasks (A, B, C) and a destination with tasks (A, B, D), both prefilling A and B. We transplant keys and values for the source task span into the destination attention computation for the selected heads. The destination still receives its original text prefix, but the intervention supplies source task information through those heads. A continuation implementing C rather than D demonstrates behavioral transfer of that information.

In the multi-predicate assignment-pivot setup, the 250-head intervention achieves approximately 99% source-directed continuation and 96% exclusive source-goal readout over four datatypes and four language directions. Source-directed continuation includes the oracle's source-or-both labels; it is not a claim of perfect exclusive replacement or executable code in every case. The cross-language conditions show that this transfer can produce a destination-language realization of source task information. Detailed configurations and failure modes are in Appendix M.1.

The implementation routes patched attention outputs to the selected query heads, accounting for shared KV heads. It applies across prefill query positions and subsequent decoding. Consequently, earlier patched outputs can alter later destination states and queries. Frozen source keys and values do not imply a fixed destination query throughout the network. Success shows adaptation to changed task information, but does not by itself locate progress information in the query.

5.3 Queries can favor a particular source subgoal

Query transfer provides a complementary intervention. In the multi-predicate setup, transferring source queries at selected heads produces approximately 94% source-directed continuation across the four language directions. This result establishes that the query states can influence which subgoal is pursued, but source progress and source target coincide in the basic setup.

A qualitative task-order contrast makes their predictions diverge. The source orders tasks (A, B, C, D) and has completed (A, B), so its next task is C. The destination orders (A, B, D, C) and has completed only A. Transferring a source state that solely specifies the completed prefix (A, B) would favor D under the destination order, while a state favoring the source target would favor C. The observed transfer favors C. This example supports goal-specific information in the query; a systematic crossed evaluation is needed to distinguish it from other context-dependent selectors. It does not show that progress information is absent from the query or establish how the next goal is computed.

5.4 Reading intermediate residual states with OVReader

The preceding experiments manipulate attention states. We also ask at which layers the residual state at a capture position supports a subgoal readout. **OVReader** applies the selected heads' value and output maps to that residual, normalizes each head's transformed vector, aggregates the results, and uses the resulting direction in the GoalScope probe (Appendix N). It uses model weights rather than a newly trained semantic decoder. Because normalization and probe generation are part of the procedure, we describe it as an OV-based readout, not a linear probe.

At the assignment operator, readout can already be strong around layer 40. Across the five capture positions, performance generally peaks around layers 48–52 and declines toward the final layer. Positions before the assignment show different emergence profiles. Layer 40 is within the selected-head layer range, so these curves do not establish that goal computation precedes all goal-head activity. They show where the representation is accessible through this readout, not its unique origin or an irreversible commitment to a goal.

Together, task-span transfer, query transfer, and OVReader show that subgoal information is available in multiple interacting components and remains sensitive to context. They leave open whether progress and target identity are separately represented, how they are combined, and which computations initially select the subgoal.

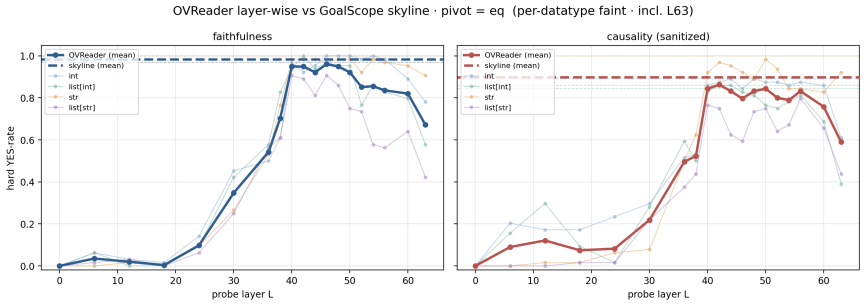


Figure 2: OVReader at the assignment pivot: readout quality across source layers, compared with the selected heads' actual output readout. The evaluation uses 64 examples per datatype. The plotted "causality" metric is implementation transfer with trivial-readout sanitization. This pre-implementation position is separated from the first implementation token; other positions appear in Appendix N.

6 Beyond Explicitly Specified Code Tasks

Our controlled corpus specifies the component predicates in the prompt. To explore whether the code-localized head set reads information beyond these explicit specifications, we examine free-form code and a few-shot relation setting. These extensions are qualitative and are kept separate from the controlled-corpus rates.

Free-form code. When a programming request does not enumerate its component steps, GoalScope can be applied to positions within the generated solution. Here the model's own continuation, rather than a preassigned predicate list, is the relevant comparison. The granularity of a subgoal can be ambiguous: a readout may describe a local operation or a larger enclosing computation. We therefore treat these examples as exploratory and defer the examples and annotation choices to Appendix J.

Few-shot relation tasks. Following the function-vector setting (Todd et al., 2024), a prompt supplies input–output demonstrations and ends at the answer prefix for a new input. The relation need not be named in the prompt. We retain the code-localized heads and ask whether their contributions expose the operation inferred from the demonstrations. Unlike sequential code, this setup tests an inferred task mapping rather than selection among explicitly listed subgoals.

```
Q: enter  A: exit
Q: feasible  A: infeasible
. . .
Q: mercy  A:
```

This abbreviated prompt is drawn from the exploratory antonym run, whose clean continuation begins "cruelty" and then continues producing question–answer pairs. At the newline following "cruelty" (saved token position 57), the readout is "Identifying antonyms." At the answer-prefix token `A` before the later answer "slow" (position 70), it is again "Identifying antonyms." These are captured during the continuing sequence, not before the first answer. Nearby positions also produce vague or incorrect descriptions, so the example establishes qualitative accessibility rather than a reliability estimate.

The probe asks for the hidden rule in a short phrase; it does not name antonymy or receive the demonstrations as text. The head set remains code-localized, but the verbalization prompt is adapted to this domain. Exact prompt wording and contrasting readouts appear in Appendix K. A quantitative evaluation should use held-out examples and a relation-description rubric, reporting clean task accuracy and unsteered probe outputs separately. We leave that evaluation pending rather than claiming quantified non-code transfer.

Additional recipe, travel, and narrative settings are reserved for Appendix K. Replications within other models, including code-trained and instruction-trained variants, are reported separately in Appendix L.1; no cross-model activation transfer is assumed.

7 Related Works

Planning and look-ahead in LMs. A growing body of work shows that autoregressive LMs encode information about tokens they have not yet emitted and act toward future constraints (Pal et al., 2023; Lindsey et al., 2025). We examine an explicit, decodable representation of the *next instrumental sub-goal*, present before execution, and evaluate both its verbalization and its use in steering code generation.

claude: Add citations: future-lens / prompt-injection look-ahead, poetry-planning (Anthropic biology), chess/maze planning, emergent world models.

Portable task representations and function abstractions. Compact, transplantable representations of a task have been found in attention-head outputs (function vectors, Todd et al., 2024), in concept-induction heads that carry language-independent word meanings (Feucht et al., 2025), and in *filter heads* that encode a filtering predicate in their *query* states and reapply it across formats, languages, and tasks (Sen Sharma et al., 2026). Our *goal heads* extend this line to *intent*: a compact, transplantable, cross-language encoding of the next instrumental sub-goal, read out and routed by a sparse head set. The filter-heads finding that models sometimes *eagerly evaluate* rather than carry a portable predicate connects to our failure-mode analysis (Section 8).

Verbalizing latents. GoalScope builds on Patchscope (Ghandeharioun et al., 2024) and related self-interpretation / lens methods, adapting them to read attention-head outputs at a single position over multi-token rollouts.

claude: Add: logit lens, tuned lens, SelfIE, function-vector readouts; contrast what GoalScope reads (head-output, multi-token) vs. residual-stream single-token lenses.

High-level interpretability of goal-directed behavior. Our work is an instance of the high-level, mechanism-agnostic interpretability agenda (Greenblatt, 2024): we build a theory of goal representation at an abstraction above the token-to-syntax mapping and validate it with intervention experiments across programming languages and replications within multiple models.

claude: Cite causal abstraction / DAS (Geiger et al.), high-level causal mediation; position our cross-language transfer as the abstraction test.

8 Discussion

The upcoming subgoal is accessible before its implementation through a readout of selected attention-head contributions. These contributions can also steer code in another language, and controlled restrictions establish causal involvement of a broader selected set in native implementation. Query and task-span interventions show that the representation interacts with context, while OVReader traces its accessibility through model depth.

Meaning and realization. Cross-language implementation transfer supports a representation that can guide more than one surface realization. Natural-language verbalization provides a complementary description of its content. Neither result establishes that the internal representation is itself natural language, or that every detail of the eventual implementation has been selected at the capture point. The position-wise evidence concerns an upcoming operation, not a complete long-horizon plan.

Causal involvement and selection. The ablations distinguish effective readout from native dependence on the selected heads. They do not establish a unique bottleneck or demonstrate adaptive rerouting. Similarly, a query that favors a source goal and a KV intervention that changes the executed goal do not identify a unique progress representation. Establishing the algorithm of subgoal selection requires contrasts that independently vary completed work, task order, and target identity while controlling intervention sites.

Scope and limitations. Our principal quantitative setting is controlled code generation with a known frontier. Oracle agreement can credit partial implementations and does not certify executable correctness. Readout, task execution, and successful transfer are therefore distinct outcomes. The five capture positions are repeated observations on each specification, and the head-budget selections are not strictly nested. Cross-model results replicate the procedure within each model; they do not show interchangeability of latent coordinates. Free-form code and non-code examples remain exploratory. Boundary-signal interventions are reported as a separate supporting direction in Appendix E.

The results provide a concrete way to examine the semantic operation a model is about to pursue. They motivate further work on when these readouts fail, how subgoals are selected without explicit decomposition, and how their representations change as computation proceeds.

Ethics

Reproducibility

Acknowledgements

References

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 1(1):12, 2021.

Sheridan Feucht, Eric Todd, Byron C Wallace, and David Bau. The dual-route model of induction. In *Conference on Language Models (COLM)*, 2025. URL https://openreview.net/forum?id=bNTrKqqnG9.

Rohit Gandikota and David Bau. Gaze heads: How vlms look at what they describe. *arXiv preprint arXiv:2606.14703*, 2026.

Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In *International Conference on Machine Learning (ICML)*, 2024.

Ryan Greenblatt. High-level interpretability: how LLMs make decisions in pursuit of goals, 2024. Interview remarks; high-level, intervention-style interpretability of goal-directed behavior.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, Joshua Batson, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

Aaron Mueller, Jannik Brinkmann, Millicent Li, Samuel Marks, Koyena Pal, Nikhil Prakash, Can Rager, Aruna Sankaranarayanan, Arnab Sen Sharma, Jiuding Sun, et al. The quest for the right mediator: Surveying mechanistic interpretability for nlp through the lens of causal mediation analysis. *Computational Linguistics*, 52(1):331–378, 2026.

Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C. Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Conference on Computational Natural Language Learning (CoNLL)*, 2023.

Arnab Sen Sharma, Giordano Rogers, Natalie Shapira, and David Bau. LLMs process lists with general filter heads. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://filter.baulab.info/. arXiv:2510.26784.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 12388–12401. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/92650b2e92217715fe312e6fa7b90d82-Paper.pdf.

A More Details on the Task Setup and Results

Full prompt + details.

Segmentation using an Oracle LLM.

Evaluation using an Oracle LLM.

B Localizing Attention Heads

Localization uses contrastive single-predicate prompts and learns a mask jointly over attention heads. The target is a token appropriate for the source predicate in the destination language, chosen to differ from the source continuation token. Applying the learned mask selects head activations for replacement. Subsequent readout uses the selected heads' projected contributions; the localization next-token objective and the multi-token evaluation are distinct.

The core Qwen2.5-Coder-32B selection has 122 heads in layers 28–55, from a thresholded mask within the candidate layer band. The broader set used in ablations contains 250 heads and includes earlier and later layers. The small-set readout sweep uses available selections from training snapshots and mask sweeps; these are not strictly nested prefixes of one ranking. Counts alone therefore do not determine head membership or the intervention's depth distribution.

Draft completion item: consolidate localization training splits, mask optimization settings, selection-file hashes, and per-model selection provenance into a final configuration table.

C GoalScope

C.1 Capture positions

The evaluation indexes the final four prompt tokens around a prefilled frontier assignment and the first generated implementation token: previous-line end, indentation, variable, assignment operator, and first implementation. Position names refer to the actual tokenized boundary; they are not interchangeable with an entire source line. The first implementation token is not a pre-implementation pivot. A cached state at a given position has no causal access to subsequent tokens, even when those tokens are displayed for context.

C.2 Aggregation and probe generation

At source position t, the main readout aggregates projected head contributions $g_t = \sum_{(\ell,j)\in H} a_t^{\ell j}$. At probe layer ℓ and current position s, norm-relative additive injection uses

$$r_{\ell,s} \leftarrow r_{\ell,s} + \alpha \|r_{\ell,s}\| \frac{g_t}{\|g_t\|}. \tag{3}$$

The main configuration uses $\alpha = .1$, layers 35–45 inclusive, and repeated injection during greedy probe generation. This coefficient is a fraction of the probe's local residual norm and is not interchangeable with an unnormalized additive coefficient. The source vector is fixed for the whole readout.

The default verbalization prompt is:

> System: You are a helpful assistant proficient in programming.
> User: What are you doing at this line of the code. Keep your answer as brief as possible.
> Assistant:

The probe receives no source code as text. Implementation transfer uses a code-stub probe instead and measures nontrivial agreement with the source subgoal. A distinct replacement variant substitutes selected heads' pre-output-projection activations in their corresponding layers, rather than aggregating them into a single direction; this is the variant used in the cross-model table. Its layer sites come from the head set, not the additive injection band.

C.3 Evaluation

An oracle judges whether the generated description or implementation matches a nontrivial part of the frontier description. Reported implementation-transfer results apply trivial-readout sanitization to avoid crediting a probe's default sign/zero comparisons. These judgments do not certify execution correctness. Clean continuation agreement, probe readout agreement, and intervention-induced changes are separate measurements. Corpus size and capture-position pooling are stated with each result.

D Gaze Mechanism

D.1 Setup

D.2 Evaluation

The goal heads look back to what they are implementing.

The LM implements where the goal heads look back to.

D.3 Gaze heads vs Concept-Copier heads

E Boundary Signals: Moving On and Staying

This supporting line of experiments asks whether signals associated with ending a subgoal can influence when the model advances, independently of the semantic identity read by GoalScope. Prompts permit a predicate to occupy one or several lines, with repeated lines beginning with the current predicate's variable. At a line boundary, the next variable therefore indicates whether the model stays with that predicate or moves to the next one.

Source/destination contrasts are constructed so the source's next variable, the destination's native next variable, and the desired counterfactual variable differ. Head-output patching is optimized to promote the counterfactual continuation, reducing direct copying of a source variable as an explanation. Transfers from completed to incomplete steps can induce moving on; reverse transfers can induce staying or repetition. Variable renaming, cross-language examples, and reversed instruction order probe the reach of this signal.

These observations motivate a distinction between subgoal identity and transitions between subgoals. A next-variable change alone does not show correct implementation of the next predicate, and repeated output under a stay intervention requires separate interpretation. Full continuation scoring and tests of changes to goal readouts are needed before claiming a complete causal dissociation. Boundary experiments are therefore supporting material rather than a premise of the main subgoal-representation account.

F Checking for Functional Dissociation

F.1 Goal heads vs Impulse heads

G Role of MLPs

arnab: TODO: path patching experiments: patch + freeze MLPs with dest values. Measure intervention effects.

H Generalization across Programming Styles and Languages

I Generalization Across LMs

arnab: replicate everything across at least 1 model different model size, and 1 different model family

J Generalization to Unspecified Settings

K Generalization to Non-Code Settings

K.1 Few-shot antonym example

The saved exploratory run in `19_goalscope_out_of_domain.ipynb` uses six demonstrations: enter/exit, feasible/infeasible, manned/unmanned, fuzzy/smooth, arise/settle, and far/near. The prompt ends at `Q: mercy` followed by `A:`. Its continuation starts with "cruelty" and then generates further question–answer pairs. The code-localized 122-head set supplies norm-relative additive readouts with $\alpha = .1$, injection layers 35–45, and repeated steering during readout generation.

The probe uses the system instruction "You are a helpful assistant." and the user instruction "You are answering a series of questions that all follow the same hidden rule. In one short phrase, what task are you performing?" This is a domain-adapted probe, not the original code-oriented prompt. Saved token indices refer to the notebook's tokenized prompt plus continuation.

Position	Source token	Readout
57	newline after cruelty	Identifying antonyms.
60	light	Identifying patterns.
70	A before slow	Identifying antonyms.
71	colon before slow	Decoding or interpreting slower actions.

Table 3: Selected qualitative readouts from one saved antonym run, including a vague description and a mismatch. These are not a quantitative evaluation sample.

K.2 Other exploratory domains

Recipe, travel, and narrative prompts use adapted verbalization questions. These settings require separate decisions about the granularity and correctness of the next step; they are not included in the code results or claimed as a validated general-purpose planning readout. A systematic evaluation remains to be completed.

L Readout Sources and Replication Across Models

Injected source	Faithfulness	Implementation transfer
122-head aggregate	.879	.792
Attention output, L40–50	.343	.334
Residual, L25–56	.031	.020
Residual, L40–50	.009	.006

Table 4: Source comparison under norm-relative injection at $\alpha = .1$ on the locked corpus, pooled over five positions. Poor raw-residual injection is a result about this readout configuration, not proof that the residual lacks subgoal information.

Selected contributions provide a substantially more effective injection direction than the tested full-attention and raw-residual aggregates. Equal relative norm controls overall perturbation magnitude, but does not ensure that a shared injection strength or aggregation rule is optimal for every source. OVReader provides a separate test of information extracted from the residual.

L.1 Replication within multiple models

Head localization and readout are repeated separately within each model. The OLMo sets have approximately matched counts, but head budgets across all models differ. These comparisons document the phenomenon across models rather than isolate causal effects of model size or training regime. Replacement uses the selected heads' own layers and ignores the additive steer-band

Model	Heads	Faithfulness	Transfer
Qwen2.5-Coder-32B	122	.823	.783
Qwen2.5-Coder-14B	88	.598	.401
Qwen2.5-32B-Instruct	108	.308	.540
OLMo-3.1-32B-Instruct (+RL)	93	.495	.689
OLMo-3.1-32B-Instruct-SFT	91	.595	.533

Table 5: Within-model replication using head replacement at $\alpha = 1$, on 253 specifications and five positions. These are different models and head selections, not activation transfers between models.

argument; a fixed additive band is therefore not the explanation for differences in this table. Scores should not be numerically conflated with the main norm-relative additive results.

M Details of Causal and Context Interventions

The native ablation evaluation uses 253 frontier assignments. Both intervention types begin at the final prompt token and continue at subsequent query positions during decoding; earlier query rows remain unmodified. For span restriction, the blocked key interval begins one token before the frontier description and extends through the prompt's final assignment token. Heads in layers 20–63 outside the spared set are blocked from that interval. For sink restriction, selected heads attend only to key position zero. The latter produces the sink value contribution and is not mathematically identical to zeroing a head.

The 122-head core and 250-head set differ in both membership and depth coverage. Random controls sample from the corresponding depth pool, not a matched count at each individual layer. For a set containing heads below layer 20, those heads are outside the span-restriction band. Thus total selected count and number of spared heads inside that band must be distinguished. The experiments test these specified interventions, not arbitrary removal of the entire complement or isolation of the selected heads from the rest of the network.

M.1 Task and query transfer

Task-span KV transfer uses per-query-head output routing to avoid transferring to every query head sharing a KV head. Task spans are padded to equal length and source keys are re-rotated to destination positions; equal lengths do not imply identical absolute starts. The KV patch is active across prefill positions and decoding. Query transfer patches the specified pivot tokens and reuses the source pivot query during the steering window (32 forward passes in the evaluation CLI, including prefill). The reported transfer configurations also amplify selected OV contributions with coefficient 2; this differs from the GoalScope probe's coefficient 0.1.

The principal multi-predicate 250-head runs report approximately 94% source-directed generation for query transfer ($n = 2048$) and 99% for KV transfer at the assignment pivot ($n = 512$), spanning four datatypes and four language directions. The latter uses no non-goal-head ablation. These rates count source-or-both oracle labels and do not imply perfect program correctness. GoalScope's patched source readout is a separate outcome. Q-filter examples exhibit readout/behavior disagreement; aggregate readout success should not be treated as a substitute for implementation success.

The qualitative task-order example in the main text follows the saved figure with unequal source/destination progress. It should not be described as an identical-progress experiment. Broader conclusions about query content require a systematic crossed design. We make no claim that these interventions demonstrate a particular set-subtraction operation.

N OVReader

For a residual state after source layer L, OVReader applies each selected head's value map and output map, including its pre-attention normalization gain. It unit-normalizes each transformed vector and averages them. This operator is based on model weights, but is not a single linear projection because of the per-head normalization. The aggregate steers a separate GoalScope probe with norm-relative coefficient .15; the selected-head-output comparison uses .1.

The evaluation samples 64 examples per datatype, with five capture positions and layer sweeps through the final layer. At the assignment operator, readout can approach the head-output reference around L40; pooled curves generally peak near L48–52. A decline at the final layer indicates reduced accessibility through this particular transformation and probe, not absence of all subgoal information. Implementation-transfer scores were rejudged with a trivial-output sanitizer; existing records should not be described as a freshly resampled corpus excluding all trivial target predicates.